

system by name. Here, we request Comic Sans with a font size of 30.

The next step is the key one: we use that font to *render* our text, which creates a graphical image of the text, called a *surface* in pygame. We supply the text we want to output, a Boolean that says whether we want our text to be anti-aliased, and a color in RGB format. Here, (0, 0, 0) indicates that we want our text to be black. Finally, using `blit()`, we draw the image of the text into the window at some (x, y) location.

This code works well to show the provided text in the window at the given location. However, if the text doesn't change, there will be a lot of wasted work done re-creating the `textSurface` on each iteration through the main loop. There are also a lot of details to remember, and you must get them all correct to draw the text properly. We can hide most of this complexity by building a class.

Creating a SimpleText Class

The idea is to build a set of methods that take care of font loading and text rendering in pygame, meaning we no longer have to remember the details of the implementation. [Listing 6-6](#) contains a new class called `SimpleText` that does this work.

File: PygameDemo8_SimpleTextDisplay/SimpleText.py

```
# SimpleText class

import pygame
from pygame.locals import *

class SimpleText():

    ❶ def __init__(self, window, loc, value, textColor):
        ❷ pygame.font.init()
        self.window = window
        self.loc = loc
        ❸ self.font = pygame.font.SysFont(None, 30)
        self.textColor = textColor
        self.text = None # so that the call to setText below will
                        # force the creation of the text image
        self.setValue(value) # set the initial text for drawing
```